

Project #2 – Task Management (Cyclic Executive)

Document reference: Thies, Nate.cmpe311.fall25.Project#2-CE

Date of publication: November 7th, 2025

LEAD ENGINEER: Nate Thies, University of Maryland Baltimore County

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION

This project implements a Cyclic Executive Task Manager for an embedded system on the Elegoo UNO R3 that performs identically to Project #1 – Asynchronous Execution. The system allows a user to independently set and update blink intervals for two LEDs via the Arduino IDE Serial Monitor. Each LED blinks continuously and independently at its last-set rate.

Unlike Project #1, which utilized asynchronous control logic via `millis()`, this project restructures the same behavior under a function-pointer-based Cyclic Executive (CE) dispatcher. The CE loops over an array of task functions—one per LED and one for serial input—executing them in a non-preemptive, round-robin fashion.

This design meets the same customer and high-level requirements as Project #1 while satisfying the additional Educational Requirement to implement a function-pointer-based cyclic executive.

REFERENCES

Ref: kiddKid.cmpe311.fall25.project#1 – Asynchronous Execution

Ref: projectTaskMgmtCE_draft.docx – Educational Requirement Specification

Ref: Project 1 – Submission.docx – Customer and high-level requirement definitions

Ref: Elegoo UNO R3 datasheet – Hardware platform reference

DEFINITIONS AND ABBREVIATIONS

See Project#1 – Asynchronous Execution for the complete glossary.

Additional term for this document:

Cyclic Executive (CE) – A fixed scheduling pattern that repeatedly dispatches tasks in a round-robin order via function pointers.

Task Function Array – An array of function pointers used by the CE to call each system task sequentially.

CUSTOMER REQUIREMENTS

- C.1 The User must be able to set the blink rate of two different LEDs.
- C.2 The User must be able to update the blink rate of each of the LEDs independently.
- C.3 Each LED must continue blinking at the last-set rate until updated by the User.
- C.4 The System must run upon an Arduino Uno R3 compatible development board.

HIGH-LEVEL TECHNICAL REQUIREMENTS

- HL.1 The System must use two LEDs connected to pins 2 and 3 of the Elegoo UNO R3.
- HL.2 The System must execute under a Cyclic Executive dispatcher implemented as a function-pointer array.
- HL.3 The System must communicate with the User via the Arduino IDE serial monitor (9600 baud).
- HL.4 The System must remain fully responsive; no blocking code or delays.
- HL.5 The User must specify LED blink intervals in milliseconds.
- HL.6 The blink rate of one LED must not affect or interfere with the other LED.

EDUCATIONAL REQUIREMENTS

- E.1 The embedded system must implement a Cyclic Executive Task Manager using a function-pointer array to dispatch all asynchronous tasks.
- E.2 The test and validation requirements must include those from Project #1 plus any additional tests necessary to confirm proper CE operation.
- E.3 The final report must be a formal design document following the Project #1 structure and conventions.

DESIGN

DESIGN PRE-REQUISITES

- Elegoo UNO R3 (Arduino-compatible ATmega328P)
- Arduino IDE v2.3.3 or later
- Two LEDs (any color)
- Two 330 Ω resistors (series limiters)
- Breadboard and jumper wires
- USB cable (for power and serial communication)

CIRCUIT WIRING

Signal	UNO Pin	Connection
LED 1 (anode +)	D2 $\rightarrow$ 330 Ω $\rightarrow$ LED $\rightarrow$ GND	Red LED recommended
LED 2 (anode +)	D3 $\rightarrow$ 330 Ω $\rightarrow$ LED $\rightarrow$ GND	Green LED recommended
Common Ground	GND rail	Both LED cathodes share ground

CODE

```
None

/*
  Project #2 - Task Management (Cyclic Executive)
  Platform: Elegoo/Arduino UNO R3 (ATmega328P)
  Serial Monitor: 9600 baud

  Summary:
  - Implements a non-preemptive, round-robin cyclic executive (CE)
    using a function-pointer task array.
  - Tasks:
    1) taskSerialInput : non-blocking serial line reader + UI state
machine
    2) taskBlinkLED1   : toggles LED1 according to its period
    3) taskBlinkLED2   : toggles LED2 according to its period
  - No delay(); all timing uses millis().
  - User-entered interval (ms) is treated as the FULL period; each
    ON and OFF lasts interval/2 milliseconds (matches P1 dialog).

  Usage (Serial):
    > What LED? (1 or 2)
    > 2
    > What interval (in msec)?
    > 600
    => LED2 blinks ~300 ms ON, ~300 ms OFF
*/

#include <Arduino.h>

// ----- Pins -----
static const uint8_t LED_PIN_1 = 2; // LED1 on D2
static const uint8_t LED_PIN_2 = 3; // LED2 on D3

// ----- Timing Model -----
// The "period" is the full blink period in milliseconds.
// Each LED toggles every halfPeriod = max(1, period/2).
// This matches the Project-1 test dialog: entering 600ms yields ~300ms
ON + ~300ms OFF.

static volatile unsigned long periodMs_LED1 = 1000; // default 1s period
static volatile unsigned long periodMs_LED2 = 1000; // default 1s period

// Internal toggling state
static unsigned long lastToggle_LED1 = 0;
```

```

static unsigned long lastToggle_LED2 = 0;
static bool state_LED1 = LOW;
static bool state_LED2 = LOW;

// ----- Serial UI State -----
// Non-blocking line input buffer
static const size_t LINE_MAX = 32;
static char lineBuf[LINE_MAX];
static size_t lineLen = 0;

enum InputState : uint8_t {
    ASK_LED = 0,
    ASK_INTERVAL = 1
};

static InputState uiState = ASK_LED;
static int selectedLED = -1; // 1 or 2 after a valid LED entry

// ----- Helpers -----
static inline unsigned long halfPeriodClamp(unsigned long period) {
    // Ensure at least 1 ms half-period to avoid zero-interval edge cases.
    unsigned long hp = period / 2;
    return (hp == 0 ? 1UL : hp);
}

static void promptAskLED() {
    Serial.println(F("What LED? (1 or 2)"));
}

static void promptAskInterval() {
    Serial.println(F("What interval (in msec)?"));
}

static bool isAllDigits(const char* s) {
    if (!s || !*s) return false;
    for (const char* p = s; *p; ++p) {
        if (*p < '0' || *p > '9') return false;
    }
    return true;
}

// Process a completed input line (no CR/LF included).
static void processLine(const char* line) {
    switch (uiState) {

```

```

case ASK_LED: {
    if ((strcmp(line, "1") == 0) || (strcmp(line, "2") == 0)) {
        selectedLED = atoi(line);
        uiState = ASK_INTERVAL;
        promptAskInterval();
    } else {
        Serial.println(F("Invalid LED. Enter 1 or 2.));
        promptAskLED();
    }
} break;

case ASK_INTERVAL: {
    if (!isAllDigits(line)) {
        Serial.println(F("Invalid interval. Must be a positive integer in
milliseconds.));
        promptAskInterval();
        return;
    }
    // Parse and clamp (0 is invalid)
    unsigned long val = strtoul(line, nullptr, 10);
    if (val == 0) {
        Serial.println(F("Invalid interval. Must be > 0.));
        promptAskInterval();
        return;
    }

    if (selectedLED == 1) {
        periodMs_LED1 = val;
        Serial.print(F("LED1 period set to "));
        Serial.print(periodMs_LED1);
        Serial.println(F(" ms (ON/OFF ~half each).));
    } else if (selectedLED == 2) {
        periodMs_LED2 = val;
        Serial.print(F("LED2 period set to "));
        Serial.print(periodMs_LED2);
        Serial.println(F(" ms (ON/OFF ~half each).));
    } else {
        // Should never happen; reset UI gracefully
        Serial.println(F("Internal state error; resetting prompt.));
    }

    // Reset for next cycle
    uiState = ASK_LED;
    selectedLED = -1;
}

```

```

        promptAskLED();
    } break;
}
}

// ----- Tasks -----
// Task 1: Non-blocking serial input reader + UI state machine.
// - Accumulates characters only when available.
// - Handles Backspace/Delete, ignores CR, ends lines on LF.
// - Never blocks; safe inside cyclic executive.
void taskSerialInput(void) {
    while (Serial.available() > 0) {
        char c = (char)Serial.read();

        if (c == '\r') {
            // Ignore CR; wait for LF to terminate line
            continue;
        } else if (c == '\n') {
            // End of line -> process
            lineBuf[lineLen] = '\0';
            if (lineLen > 0) {
                processLine(lineBuf);
            }
            lineLen = 0; // reset buffer
        } else if (c == 8 || c == 127) { // Backspace or Delete
            if (lineLen > 0) {
                lineLen--;
            }
        } else {
            // Accept visible characters up to buffer limit
            if (lineLen < (LINE_MAX - 1)) {
                lineBuf[lineLen++] = c;
            }
            // else: silently drop extra chars to remain non-blocking
        }
    }
}

// Task 2: Blink LED1 according to its configured period.
void taskBlinkLED1(void) {
    unsigned long now = millis();
    unsigned long hp = halfPeriodClamp(periodMs_LED1);
    if ((now - lastToggle_LED1) >= hp) {
        state_LED1 = !state_LED1;
    }
}

```

```

        digitalWrite(LED_PIN_1, state_LED1 ? HIGH : LOW);
        lastToggle_LED1 = now;
    }
}

// Task 3: Blink LED2 according to its configured period.
void taskBlinkLED2(void) {
    unsigned long now = millis();
    unsigned long hp = halfPeriodClamp(periodMs_LED2);
    if ((now - lastToggle_LED2) >= hp) {
        state_LED2 = !state_LED2;
        digitalWrite(LED_PIN_2, state_LED2 ? HIGH : LOW);
        lastToggle_LED2 = now;
    }
}

// ----- Cyclic Executive -----
typedef void (*TaskFunc)(void);

// Round-robin task list (order can be adjusted)
TaskFunc taskList[] = {
    taskSerialInput,
    taskBlinkLED1,
    taskBlinkLED2
};

static const int NUM_TASKS = (int)(sizeof(taskList) /
sizeof(taskList[0]));

void setup() {
    pinMode(LED_PIN_1, OUTPUT);
    pinMode(LED_PIN_2, OUTPUT);

    digitalWrite(LED_PIN_1, LOW);
    digitalWrite(LED_PIN_2, LOW);

    Serial.begin(9600);
    while (!Serial) { /* wait for native USB; UNO returns immediately */ }

    Serial.println(F("System ready (Cyclic Executive)."));
    promptAskLED();
}

void loop() {

```

```
// Non-preemptive round-robin dispatcher
for (int i = 0; i < NUM_TASKS; ++i) {
    taskList[i]();
}
// No delay(); fast polling keeps everything responsive.
}
```

TESTING AND VALIDATION

T.0 through T.5.2 were conducted successfully on the Elegoo UNO R3 hardware setup using two LEDs and 330 Ω resistors. All behavioral tests confirmed correct non-blocking performance and independence of LED control under the Cyclic Executive task manager.

APPENDIX A – LOGIC FLOW

Setup → Initialize LEDs, Serial → Start Cyclic Executive Loop →
taskSerialInput → taskBlinkLED1 → taskBlinkLED2 → Repeat

APPENDIX B – DESIGN CODE

Refer to the provided Arduino sketch for Project #2 – Task Management (Cyclic Executive).

DEMO VIDEO

https://drive.google.com/file/d/1qj3VP4y6h0wDKOSP71zRo_uxsp1ne4BW/view?usp=sharing